AI로 무엇을 만들었나

주문 · 결제 · 상품 · 회원

발표자 2 · 7분

AI로 쇼핑몰 만들기 — 잘 묻고, 반드시 검증한다

주문은 “구매 버튼 누르면 끝”이 아닙니다

- 재고 확인 · 차감
- 쿠폰 할인 · 적립금 사용
- 배송비 계산
- 주문 생성 · 결제 대기
- **중간에 하나라도 실패하면 전부 롤백**

> 이런 건 바로 코드부터 뽑으면 안 된다

그래서 “접근 먼저” 로 물었습니다

```
[맥락] Medusa v2 백엔드. 한국 쇼핑몰 주문 흐름 설계.

[할 일] 장바구니 → 결제 대기까지의 워크플로우
  1) 재고 확인·차감 (동시 주문에도 안전하게)
  2) 쿠폰 할인    3) 적립금 사용
  4) 배송비       5) 주문 생성    6) 결제 대기
  * 중간 실패 시 앞 단계 전부 롤백

[진행] 바로 코드 짜지 말고, 단계별 흐름과
  입력·출력·실패 시 보상을 표로 제안해.
  내가 확인하면 구현.
```

돌아온 건 코드가 아니라 흐름표

```
1 장바구니 검증 → 2 재고 확인(동시성) → 3 할인 →
4 적립금 차감 → 5 배송비 → 6 주문 생성 →
7 재고 예약 → 8 결제 정보

각 단계 실패 시 앞 작업을 자동 보상
```

> "여기 순서 이상한데?"를 우리가 짚을 수 있었다. 코드가 나온 뒤였으면 다 갈아엎어야 했다

걱정되면 근거를 요구했습니다

동시 주문

재고 1개인데 두 명이 동시에 주문하면?

추측하지 말고 Medusa 재고 처리 코드를
확인해서 파일:라인으로 근거를 대줘.

할인 순서

쿠폰 10% + 적립금 5000원을 같이 쓰면
어느 걸 먼저 적용해야 하지?

순서에 따라 최종 금액이 달라지는지
계산해서 보여줘.

> "안전합니다"만 받았으면 넘어갔을 것 — 근거를 요구하니 실제 코드를 확인하고 답했다

한 프롬프트 = 한 기능

"주문 워크플로우 전체 만들어줘"는 실패했다

그래서 쪼갰습니다

- **1단계** 재고 확인만
- **2단계** 할인 계산만
- **3단계** 적립금 차감만
- **4단계** 합치기

> 각 단계마다 테스트하고 넘어갈 수 있었다.
> 통째로 던지면 그럴듯하지만 검증 불가능한 코드가 나온다

1단계 프롬프트

```
[할 일] 재고 확인 함수 하나만. 다른 단계는 아직 X.
[패턴] 옆에 있는 validateCart 함수 구조 그대로.
[출력] 타입 먼저 제안하고, 확인받은 뒤 구현.
[검증] 동시 요청 2건 테스트 코드도 같이.
```

상품·회원 — “옆에 있는 저것처럼”

❌ **알아서 잘**

상품 목록에 가격 필터 추가해줘

✅ **기존 코드를 지목**

상품 목록에 가격 필터 추가해줘.
옆에 있는 카테고리 필터(refinement-list/)
구조 그대로 따라가고,
URL 쿼리 파라미터 방식도 동일하게.

> 커스텀 모듈 27개가 전부 같은 구조 — 하나만 이해하면 나머지가 읽힌다

고치는 과정에서 다른 걸 깨뜨렸습니다

- **보안 패치가 린터를 죽였다**
  취약점 대응으로 넣은 버전 고정이 ESLint 가 쓰는 라이브러리까지 올려서, 린트가 아예 안 돌았다

- **첫 설계가 프레임워크에 막혔다**
  재고 강제 차감을 `?force=true` 쿼리로 만들었더니 코어 검증에 걸려 400. 헤더 방식으로 바꿔 해결

- **최적화를 켰더니 페이지가 죽었다**
  이미지 최적화 ON → 등록 안 된 이미지 호스트 때문에 상품 상세가 500

> 셋 다 **실제로 돌려보고** 알았습니다. 그래서 "끝나면 검증까지"가 프롬프트에 들어갔습니다

정리

- **복잡한 건 설계부터** — 코드 전에 흐름표
- **걱정되면 근거를 요구** — "추측하지 마"
- **한 프롬프트 = 한 기능** — 쪼개야 검증할 수 있다
- **고치면 옆이 깨진다** — 그래서 매번 돌려봐야 한다